Pipelined Processor Architecture

CIE 249: Computer Architecture and Assembly Language

Instructor: Elmahdy Maree

By: Team Number One

Contents

1 Introduction to Pipelining

Pipelining is a fundamental architectural optimization designed to maximize processor throughput. By subdividing the execution cycle of a traditional single-cycle processor into five distinct, overlapping stages, multiple instructions can be processed concurrently. Since each individual stage handles roughly one-fifth of the overall logic, the processor's clock frequency can be increased proportionally.

While the latency of an individual instruction remains relatively unchanged, the aggregate throughput (instructions completed per second) improves dramatically. For example, a single-cycle processor might require 680 ps per instruction, achieving a throughput of 1.47 billion instructions per second. In contrast, a pipelined processor clocked at 200 ps has an instruction latency of 1000 ps (5×200 ps) but completes a new instruction every 200 ps, yielding an extraordinary throughput of 5 billion instructions per second. Due to this immense advantage, pipelining is universally employed in modern high-performance microprocessors.

2 Pipeline Stages and Architecture

A processor's execution cycle is depending on its slowest operations which is around 200 ps: accessing memory, reading the register file, and performing Arithmetic Logic Unit (ALU) computations. To optimize performance, the processor is divided into five stages, isolating these slow operations:

1. **Fetch (F):** The processor retrieves the instruction from instruction memory and increasing `PC` register by 4.
2. **Decode (D):** The register file is accessed to read source operands, and the control unit decodes the instruction and increasing `R15` register by 8.
3. **Execute (E):** The ALU executes mathematical computations or calculates memory addresses.
4. **Memory (M):** Data is either read from or written to the data memory.
5. **Writeback (W):** The final computed result is written back into the register file.

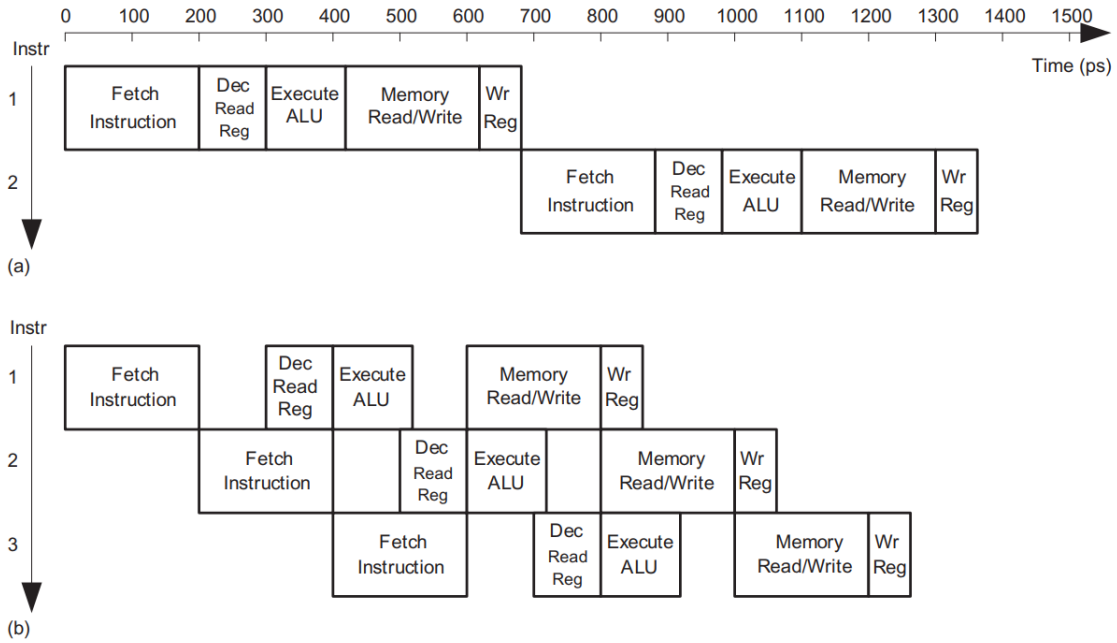

Figure 1: Timing diagrams: (a) single-cycle processor and (b) pipelined processor

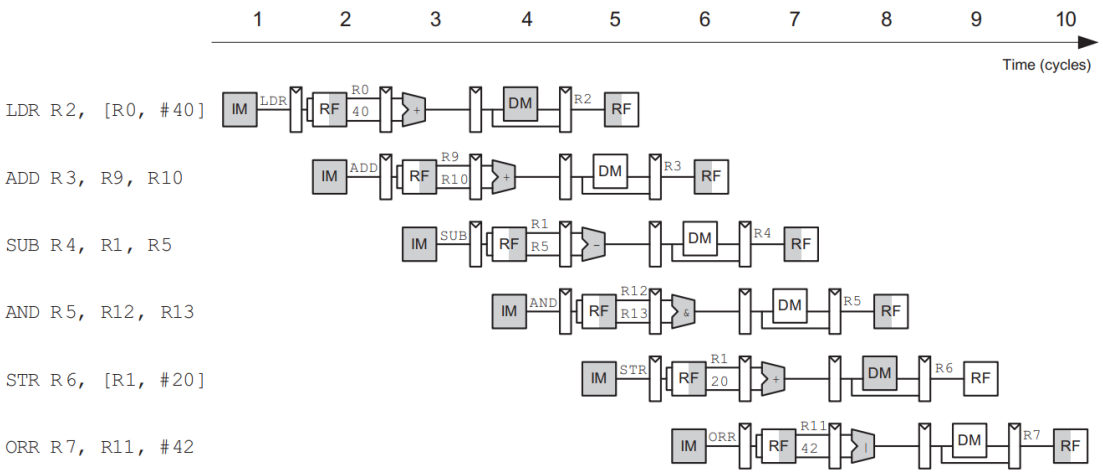

Figure 2: Abstract view of pipeline in operation

Figure 2 illustrates the parallel nature of pipelining. A key hardware optimization enabling this flow involves the register file: it writes on the falling edge of the clock (`CLK`). This allows the processor to write a result during the first half of a clock cycle and read that exact result during the second half of the same cycle.

Register File Write on Falling Edge (`regfile.sv`)

```
1    always @(negedge clk) begin
2        if (we3 && (a3 != 4'd15)) rf[a3] <= wd3;
3    end
```

3 The Pipelined Datapath and Control Unit

3.1 Datapath Construction

To physically implement pipelining, four sets of pipeline registers are inserted into the standard datapath, isolating the five stages. These registers ensure that intermediate data and signals are preserved as an instruction advances to the next cycle.

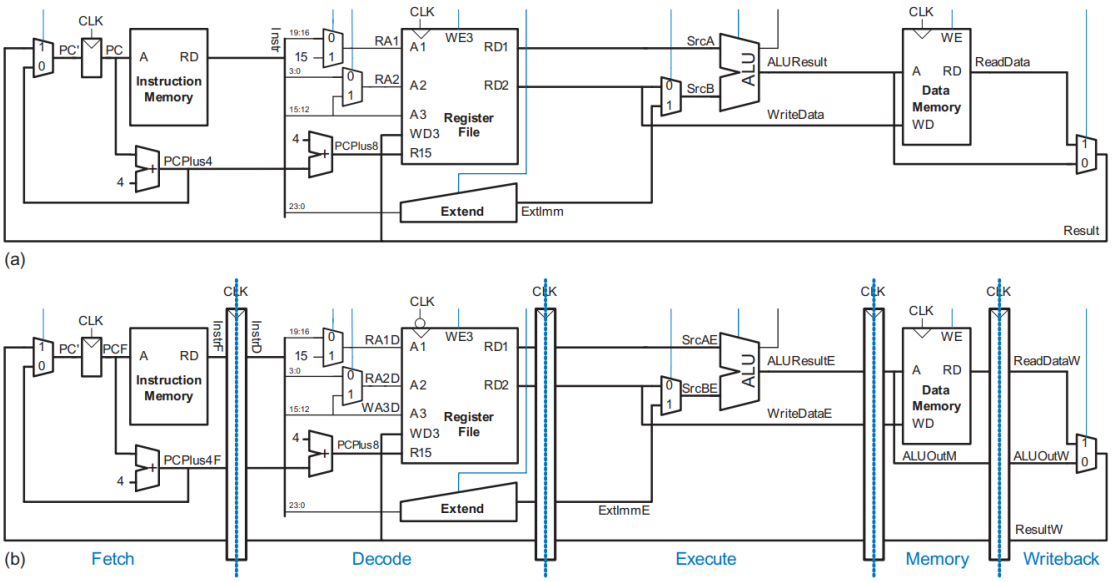

Figure 3: Datapaths: (a) single-cycle and (b) pipelined

A critical challenge in datapath design is signal synchronization. For example, the destination register write address (`WA3`) must travel through the pipeline registers sequentially. If `WA3` was used directly from the Decode stage during a write, the processor would overwrite the wrong register, corrupting the state of an older instruction.

Propagating `WA3` Through Pipeline Registers (`datapath.sv`)

```
1    // D->E Pipeline
2    flopenrc #(.WIDTH(4)) wa3DE (.clk(clk), .reset(reset), .en(1'b1), .clear(
         FlushE), .d(InstrD[15:12]), .q(WA3E));
3    // E->M Pipeline
4    flopr #(.WIDTH(4))    wa3EM (.clk(clk), .reset(reset), .d(WA3E), .q(WA3M));
5    // M->W Pipeline
6    flopr #(.WIDTH(4))    wa3MW (.clk(clk), .reset(reset), .d(WA3M), .q(WA3W));
```

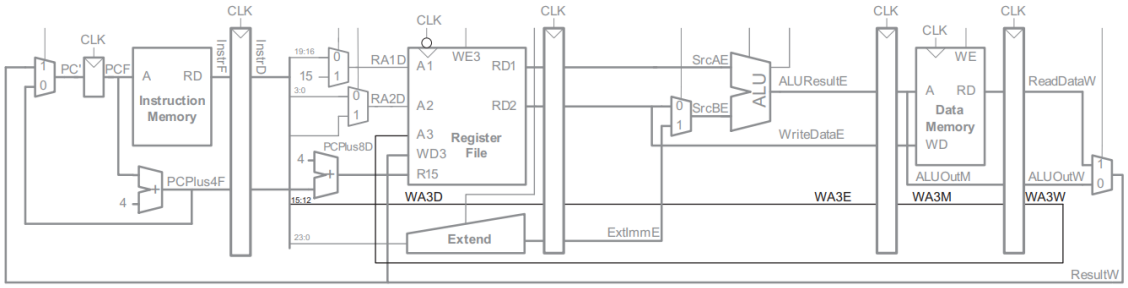

Figure 4: Corrected pipelined datapath with proper signal synchronization

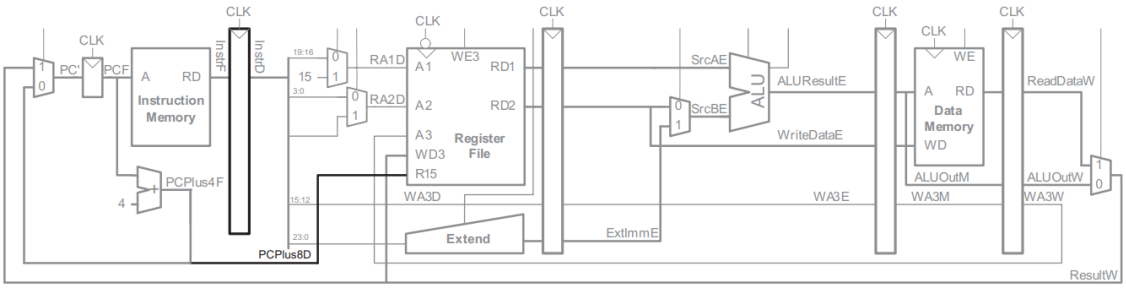

Figure 5: Optimized PC logic eliminating a register and adder

The datapath can also be simplified. The Program Counter (PC) increments by 4 over consecutive cycles. Thus, the `PCPlus4F` signal in the Fetch stage is logically equivalent to `PCPlus8D` in the Decode stage. Forwarding this signal eliminates the need for an expensive, redundant 32-bit adder (Figure 5).

Optimized PC Logic (`datapath.sv`)

```
1    assign PCPlus8D = PCPlus4F;
```

3.2 Pipelined Control Signals

Just as data must be pipelined, control signals generated by the control unit in the Decode stage must also advance through the pipeline.

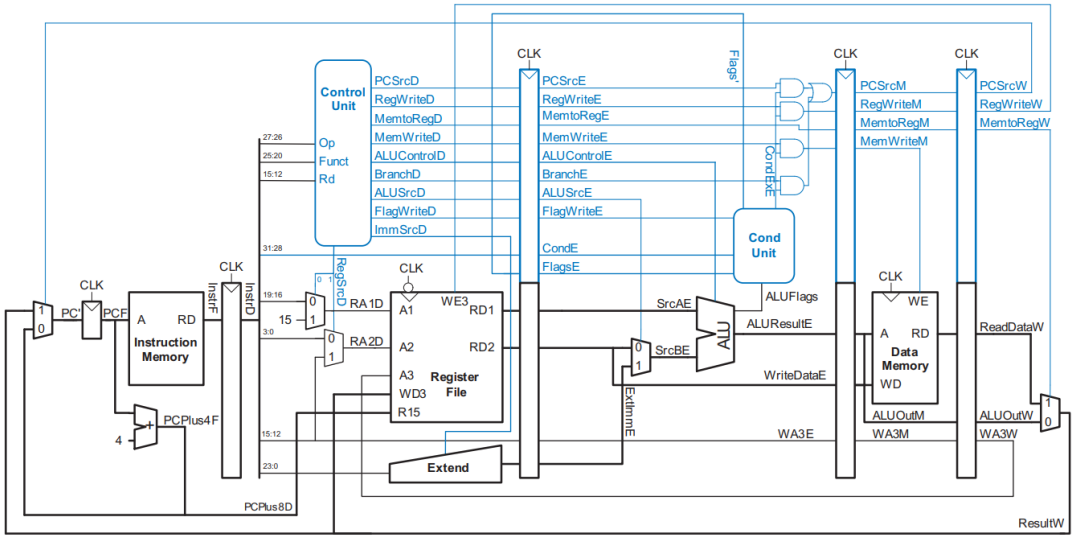

Figure 6: Pipelined processor with control unit integration

For example, the `RegWrite` signal must traverse the Execute and Memory stages, safely arriving at the Writeback stage exactly when its corresponding instruction is ready to commit data to the register file (Figure 6).

Propagating `RegWrite` Signal (`datapath.sv`)

```
1    // D->E Pipeline
2    flopenrc #(.WIDTH(1)) regwriteDE(.clk(clk), .reset(reset), .en(1'b1), .
         clear(FlushE), .d(RegWriteD), .q(RegWriteE));
3    // E->M Pipeline
```

```
4       flopr #(.WIDTH(1))     regwriteEM (.clk(clk), .reset(reset), .d(
            RegWriteE_eff), .q(RegWriteM));
5       // M->W Pipeline
6       flopr #(.WIDTH(1))     regwriteMW (.clk(clk), .reset(reset), .d(RegWriteM),
            .q(RegWriteW));
```

4 Data Hazards and Resolution Mechanisms

In a pipelined system, a **hazard** arises when an instruction depends on a result from a previous instruction that has not yet completed.

4.1 Read After Write (RAW) Hazards

A RAW data hazard occurs when an instruction attempts to read a register before an older instruction updates it. For instance, if an `ADD` writes to `R1`, and an `AND` immediately reads `R1`, the `AND` operation will incorrectly fetch the old value.

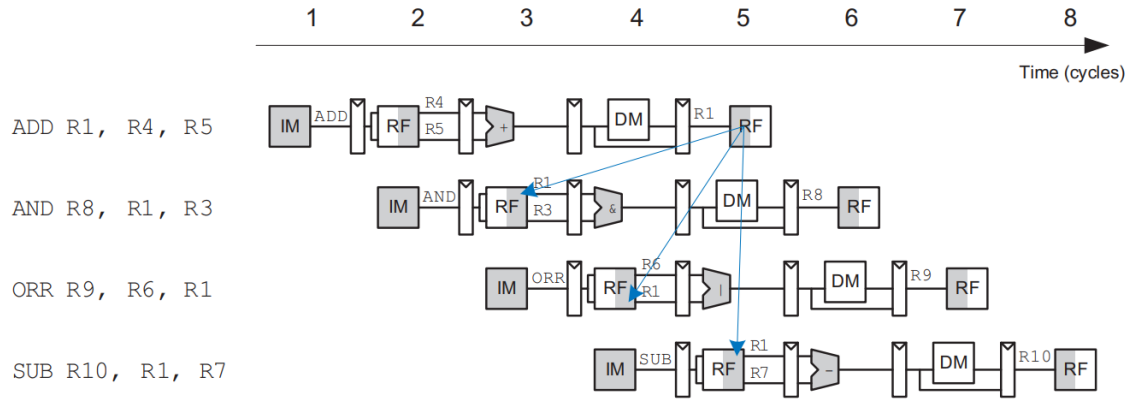

Figure 7: Abstract pipeline diagram illustrating data hazards

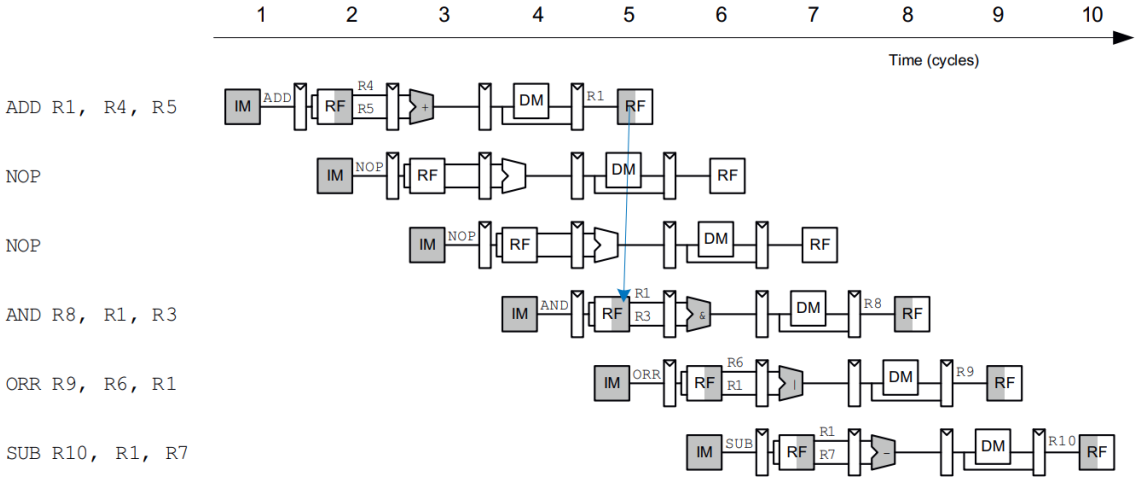

Figure 8: Solving data hazard using software NOP insertion

A rudimentary software solution is inserting blank `NOP` (No Operation) instructions to manually delay execution (Figure 8). However, this heavily degrades performance by wasting active clock cycles.

4.2 Hardware Forwarding (Bypassing)

The standard hardware solution is **forwarding**. Although the result of an `ADD` is not written until the Writeback stage, the actual computation finishes in the Execute stage. We can route this computed value directly to the Execute stage of the next instruction.

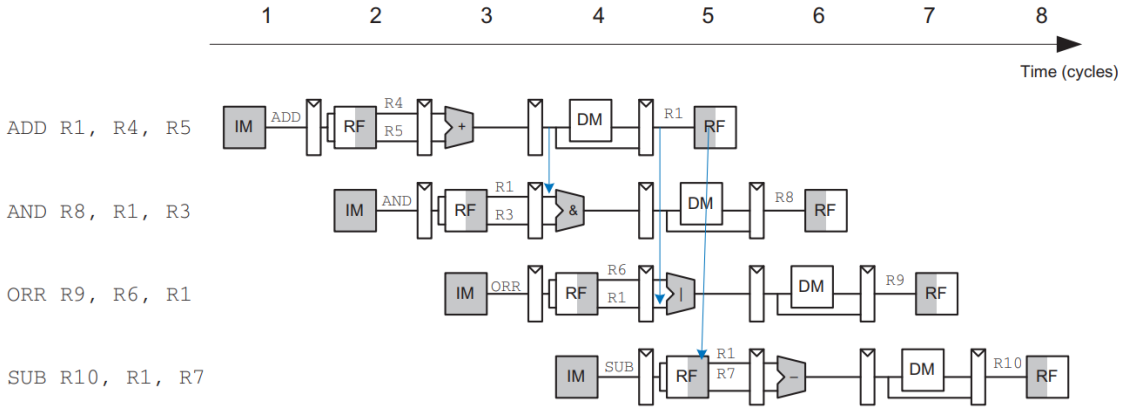

Figure 9: Abstract pipeline diagram illustrating forwarding

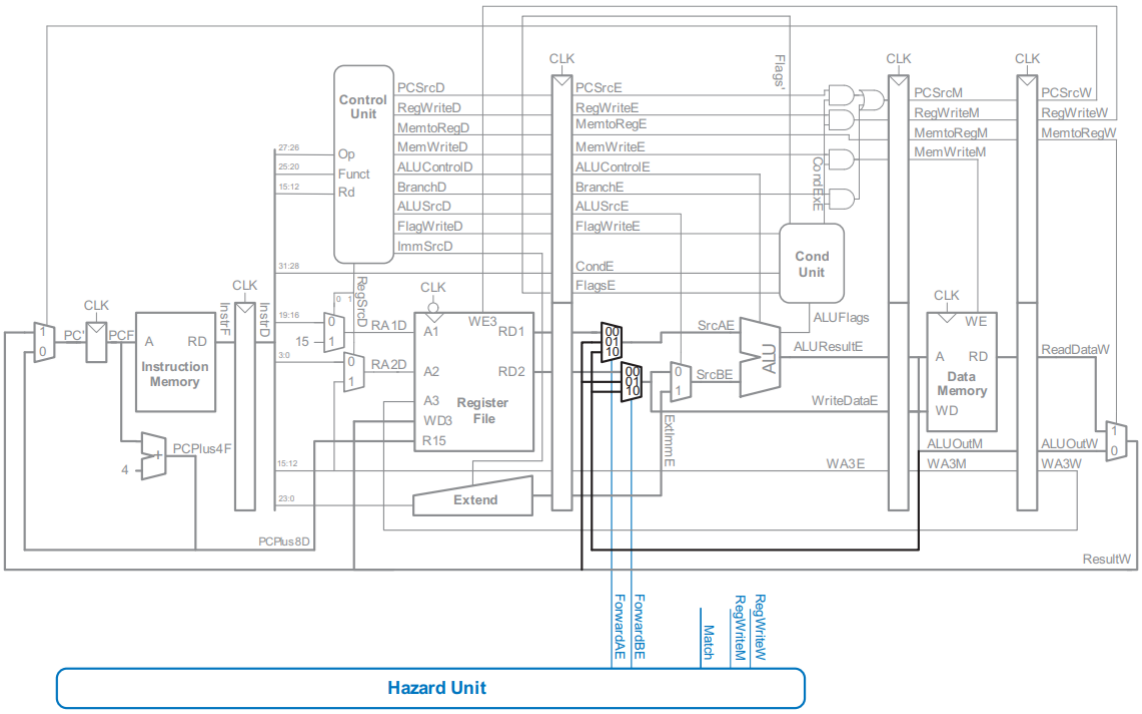

Figure 10: Pipelined processor with forwarding logic for hazard resolution

Forwarding is implemented by placing multiplexers before the ALU (Figure 10). The Hazard Unit detects if the Execute stage's source registers (`RA1E`, `RA2E`) match the destination registers of instructions in the Memory (`WA3M`) or Writeback (`WA3W`) stages. If both stages present a match, the Memory stage receives priority since its data is the most recently updated.

Forwarding Match Logic (`hazard_unit.sv`)

```
    assign Match_1E_M = (RA1E == WA3M);
    assign Match_1E_W = (RA1E == WA3W);
```

```
    assign Match_2E_M = (RA2E == WA3M);
    assign Match_2E_W = (RA2E == WA3W);

    always_comb begin
        if (Match_1E_M & RegWriteM)      ForwardAE = 2'b10;
        else if (Match_1E_W & RegWriteW) ForwardAE = 2'b01;
        else                             ForwardAE = 2'b00;

        if (Match_2E_M & RegWriteM)      ForwardBE = 2'b10;
        else if (Match_2E_W & RegWriteW) ForwardBE = 2'b01;
        else                             ForwardBE = 2'b00;
    end
```

Forwarding Multiplexers (`datapath.sv`)

```
    mux3 #(.WIDTH(WIDTH)) fwdA_mux (.d0(RD1E), .d1(ResultW), .d2(ResultM), .s(
        ForwardAE), .y(SrcAE));
    mux3 #(.WIDTH(WIDTH)) fwdB_mux (.d0(RD2E), .d1(ResultW), .d2(ResultM), .s(
        ForwardBE), .y(SrcBE_raw));
```

4.3 Load-Use Hazards and Stalling

Forwarding cannot solve all dependencies. A load instruction (`LDR`) has a two-cycle latency because it does not fetch memory data until the end of the Memory stage. If the subsequent instruction needs this data immediately, forwarding cannot traverse backward in time.

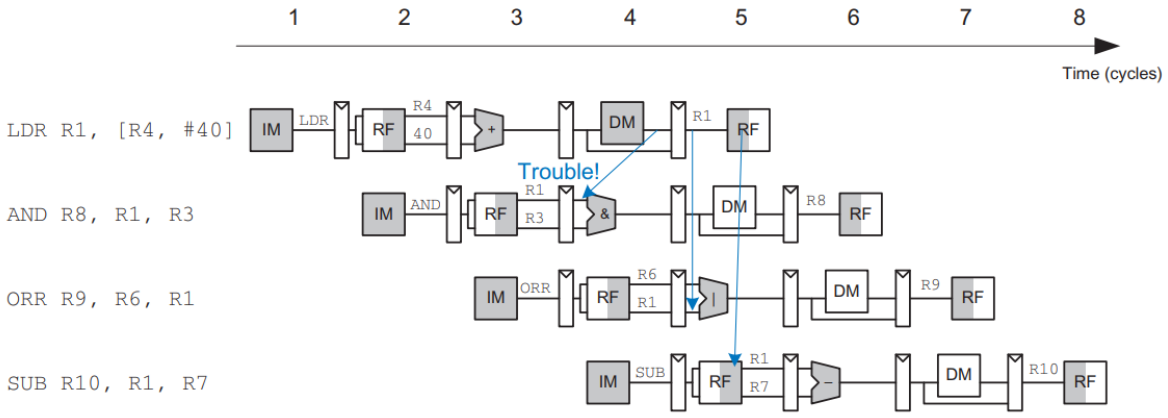

Figure 11: Abstract pipeline diagram illustrating LDR load-use hazard

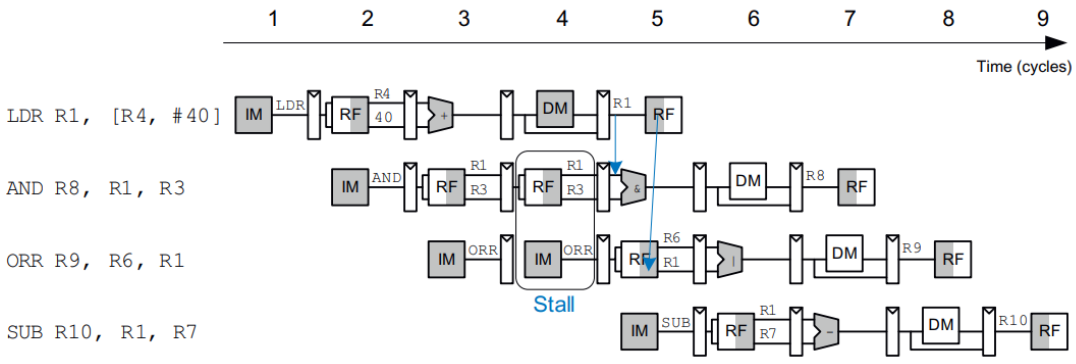

Figure 12: Abstract pipeline diagram illustrating stall insertion to resolve hazards

This load-use hazard requires a **stall**. The processor holds the dependent instruction in the Decode stage and prevents the Fetch stage from advancing. Simultaneously, the Execute pipeline register is cleared (`FlushE`), passing a harmless "bubble" down the pipeline (Figure 12).

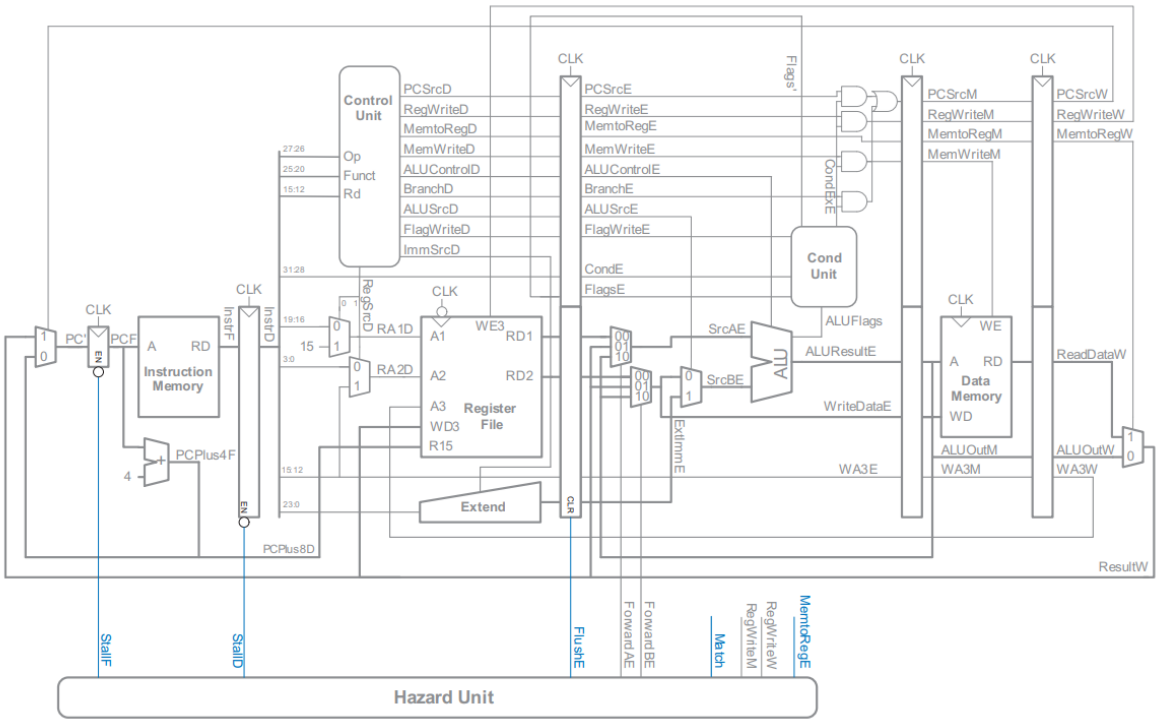

Figure 13: Pipelined processor with stall control for LDR data hazard

Stall Control Logic (`hazard_unit.sv`)

```
    assign Match_12D_E = (RA1D == WA3E) | (RA2D == WA3E);
    assign LDRstall    = Match_12D_E & MemtoRegE;

    assign StallD = LDRstall;
    assign StallF = LDRstall | PCWrPendingF;
    assign FlushE = LDRstall | BranchTakenE;
```

5 Control Hazards and Flushing

Control hazards occur during branching, as the processor fetches subsequent instructions before the branch condition is definitively resolved.

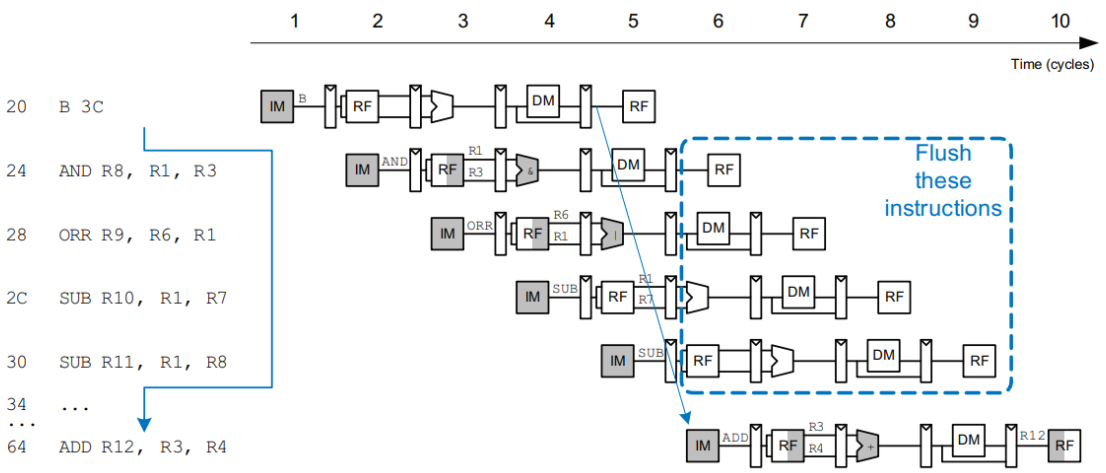

Figure 14: Abstract pipeline diagram illustrating branch flushing (control hazard)

If a branch is predicted incorrectly and is evaluated in the Writeback stage, the processor suffers a four-cycle misprediction penalty, requiring four incorrectly fetched instructions to be **flushed** (discarded).

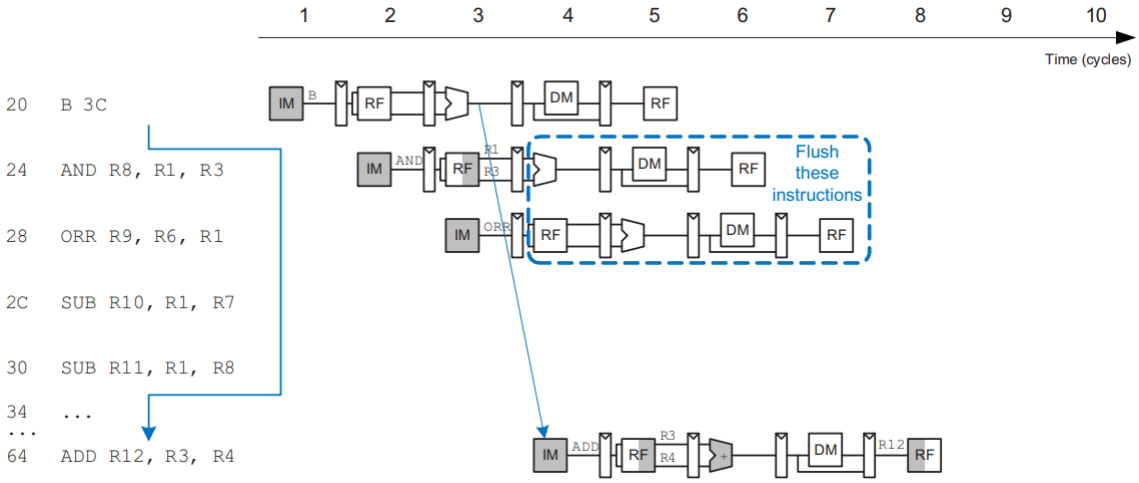

Figure 15: Abstract pipeline diagram illustrating early branch resolution

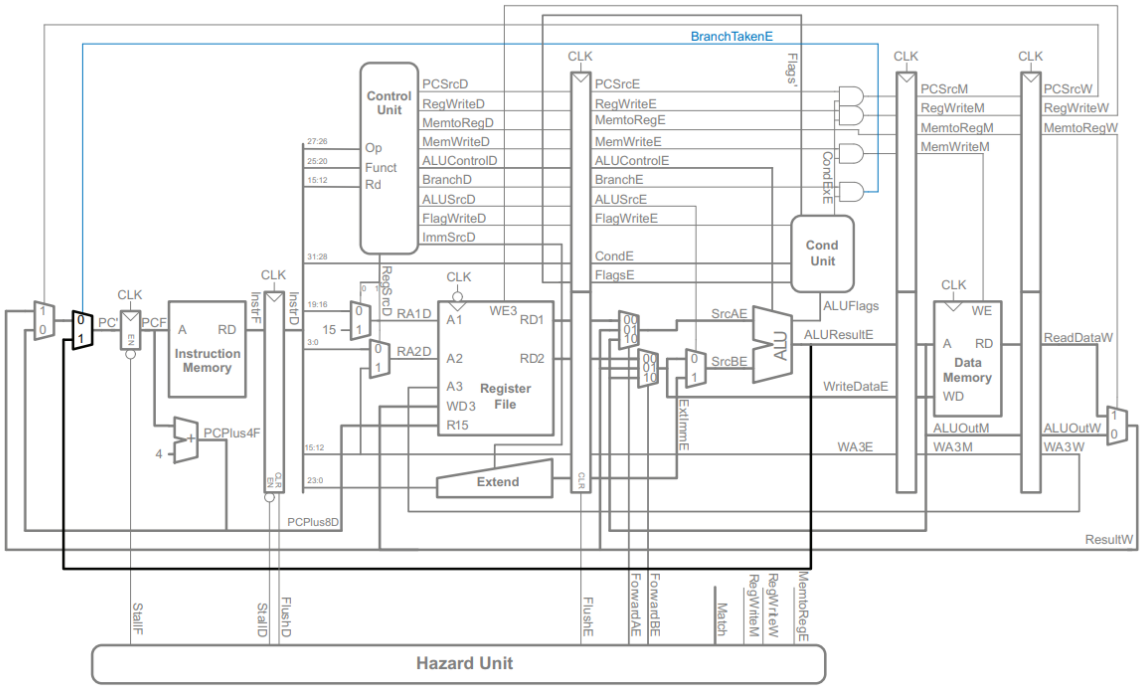

Figure 16: Pipelined processor with branch control hazard handling

To mitigate this severe penalty, branch evaluation is moved up to the Execute stage. When `BranchTakenE` is asserted early, the flush penalty is reduced to only two cycles (Figure 15). Writes to the `PC` also pause execution using the `PCWrPendingF` stall logic.

Control Hazard Flush Logic (`hazard_unit.sv`)

```
    assign PCWrPendingF = PCSrcD | PCSrcE | PCSrcM;

    assign StallF = LDRstall | PCWrPendingF;
    assign FlushD = PCWrPendingF | PCSrcW | BranchTakenE;
    assign FlushE = LDRstall | BranchTakenE;
```

6 The Complete Hazard Unit

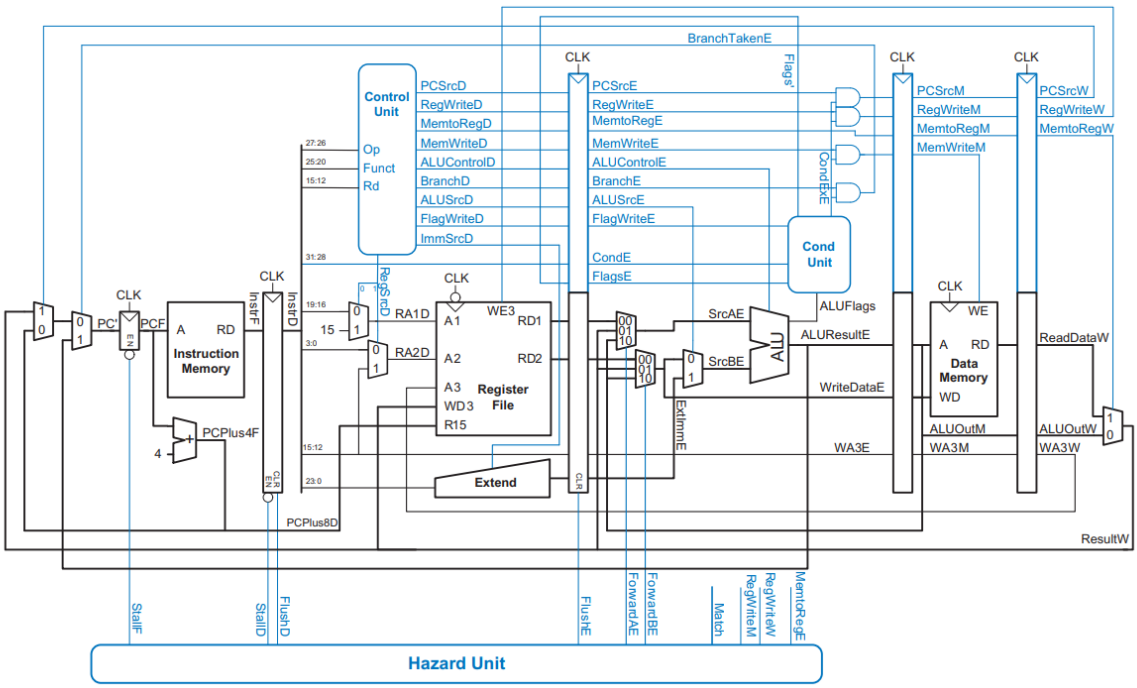

Figure 17: Complete pipelined processor with full hazard handling (forwarding, stalls, flushing)

The overarching complexity of pipelining lies in detecting and resolving instruction overlap. The Hazard Unit (Figure 17) continuously monitors instruction flow, seamlessly injecting forwards, stalls, and flushes. This elegant logic successfully orchestrates theoretical multi-stage parallelization into a robust, high-performance physical architecture capable of executing interconnected code sequences flawlessly.